



Segurança no desenvolvimento de Software

Fevereiro 2026

Carlos Castro

A Importância da Segurança no Desenvolvimento de Software

Vivemos numa era de dependência digital, onde aplicações móveis gerem as nossas finanças, plataformas web armazenam dados pessoais, e sistemas críticos controlam infraestruturas essenciais como saúde, energia e transportes. Esta transformação digital trouxe eficiência e comodidade, mas também expôs uma vulnerabilidade: os ciberataques aumentam em frequência a cada ano. As consequências de falhas de segurança podem ser devastadoras, como demonstram casos como o do **Facebook**, que expôs informação de **533 milhões de utilizadores**, ou a violação da **Equifax**, que comprometeu dados financeiros de outros **147 milhões**. Para os utilizadores afetados, as consequências podem incluir roubo de identidade, perdas financeiras diretas e violação da privacidade pessoal.

A responsabilidade pela segurança não pode ser delegada exclusivamente a equipas de cyber-segurança. Cada programador tem o dever de escrever código seguro desde o início, validando inputs, implementando autenticação robusta e prevenindo vulnerabilidades comuns como SQL injection. Esta mentalidade é reforçada pela pressão regulatória e exigências do mercado: o RGPD estabelece regras estritas com multas que podem atingir milhões de euros, enquanto empresas multinacionais exigem certificações de segurança. Desenvolver software seguro deixou de ser um diferencial competitivo para se tornar um requisito mínimo de entrada no mercado.

Enquadramento dos temas abordados

Entre as ameaças mais críticas, destacam-se três categorias que serão exploradas neste trabalho: o **SQL Injection**, as **quebras de autenticação** e a exploração de **entidades externas XML**. Estas vulnerabilidades aparecem no OWASP Top 10, um documento de referência que lista os riscos de segurança mais críticos em aplicações web.

Compreender a natureza destas ameaças, as suas consequências práticas e as técnicas de prevenção são essenciais para qualquer programador que pretenda desenvolver aplicações robustas e seguras num ambiente digital cada vez mais perigoso.

Injeção de SQL

A injeção de código SQL representa uma das vulnerabilidades mais críticas e prevalentes em aplicações web, ocorrendo quando dados não confiáveis são incorporados em comandos ou *queries*. O exemplo mais comum é a SQL Injection, onde atacantes manipulam inputs para executar comandos SQL maliciosos na base de dados. Durante os exercícios em C#, implementámos cenários que demonstraram como a concatenação direta de *strings* em *queries* SQL permite ultrapassar o processo de autenticação através de inputs como ' OR '1'='1. Esta técnica pode expor toda a base de dados, permitindo acesso e roubo de informação sensível ou até eliminação de dados críticos.

A mitigação desta vulnerabilidade passa por três abordagens principais: uso de *queries* “parametrizadas” com SqlParameter, utilização de LINQ para abstração segura das queries, e validação rigorosa de todos os inputs do utilizador. Nos exercícios em C#, para pesquisa de produtos, comparámos diretamente métodos inseguros com implementações seguras mostrando como os parâmetros SQL isolam o input do utilizador da estrutura da *query*, prevenindo injeção maliciosa. A validação de entrada adiciona uma camada extra de proteção, verificando tipos de dados e rejeitando inputs suspeitos antes mesmo de chegarem à base de dados, como demonstrado no exercício de validação de IDs onde inputs não numéricos eram rejeitados.

Quebras de Autenticação

As quebras de autenticação constituem falhas na gestão de identidade e sessões que permitem a atacantes comprometer passwords, tokens ou explorar vulnerabilidades para assumir identidades de utilizadores legítimos. Problemas comuns incluem políticas de passwords fracas, ausência de autenticação multi-fator, sessões que não expiram adequadamente e tokens previsíveis ou mal geridos. Nos exercícios práticos de SQL Injection, embora o foco principal fosse a injeção de código, também se evidenciou como falhas de autenticação podem ser exploradas: o método “*LoginInseguro*” permitia bypass completo da verificação de credenciais, dando acesso administrativo sem conhecer nenhuma password válida.

A proteção contra quebras de autenticação exige uma abordagem robusta: implementação de políticas de passwords com requisitos de complexidade e renovação periódica, uso obrigatório de autenticação multi-fator, gestão segura de sessões com timeouts apropriados, armazenamento de passwords com algoritmos de hashing seguros (como *bcrypt* ou *Argon2*), e garantia de que toda a comunicação com

credenciais ocorre exclusivamente sobre HTTPS. A combinação destas técnicas cria uma defesa em profundidade que reduz significativamente o risco de comprometimento de contas.

Entidades Externas XML

As vulnerabilidades relacionadas com entidades externas XML ocorrem quando aplicações processam documentos XML de forma insegura, permitindo que atacantes explorem *parsers* mal configurados para aceder a ficheiros do sistema, executar pedidos server-side, realizar ataques de negação ou, em casos extremos, executar código remotamente. Esta vulnerabilidade aproveita-se de uma funcionalidade legítima do **XML** que permite a definição de entidades externas capazes de referenciar recursos locais ou remotos. Por exemplo, um atacante pode submeter XML contendo `<!ENTITY xxe SYSTEM "file:///etc/passwd">` para aceder a ficheiros sensíveis do servidor, ou referenciar URLs externas para realizar ataques SSRF que contornam firewalls e acedem a recursos internos da rede.

A prevenção de ataques XXE requer desabilitar completamente o processamento de entidades externas nos parsers XML, que constitui a medida mais eficaz e recomendada. Se possível, deve-se preferir formatos de dados menos complexos como JSON, que não sofrem desta classe de vulnerabilidades. Embora os exercícios práticos realizados tenham focado principalmente em SQL Injection, os mesmos princípios de validação de entrada e uso de APIs seguras aplicam-se igualmente ao processamento de XML e outros formatos de dados estruturados.

Conclusão

A realização destes exercícios práticos sobre segurança no desenvolvimento de software evidenciou que as vulnerabilidades mais perigosas não resultam de falhas tecnológicas complexas, mas sim de práticas básicas negligenciadas durante a programação. A facilidade com que conseguimos explorar SQL Injection nos métodos inseguros, contornando autenticação e acedendo a dados não autorizados através de simples manipulações de input, demonstra que a segurança negligenciada. A diferença entre uma aplicação vulnerável e uma aplicação segura é frequentemente baseada em escolhas simples: usar parâmetros SQL em vez de concatenação de strings, validar inputs antes de processá-los, ou desabilitar funcionalidades perigosas em parsers XML.

Os principais pontos retidos centram-se em três pilares fundamentais: validação rigorosa de todos os inputs do utilizador, utilização de APIs seguras como LINQ e *queries* parametrizadas, e compreensão de que segurança é um processo contínuo. As principais aprendizagens centram-se em três pilares fundamentais: validação rigorosa de todos os inputs do utilizador, utilização de APIs seguras como LINQ e *queries* parametrizadas, e compreensão de que segurança é um processo contínuo. Estes exercícios demonstraram na prática como implementar proteções básicas que, embora simples, são eficazes contra ataques reais. A aplicação destas técnicas em projetos futuros permitirá desenvolver software mais robusto e reduzir significativamente o risco de vulnerabilidades críticas em produção.

Referências

OWASP Foundation. (2021). A03:2021 – Injection:

- https://owasp.org/Top10/2021/A03_2021-Injection/

OWASP Foundation. (2021). A07:2021 – Identification and Authentication Failures.

- https://owasp.org/Top10/2021/A07_2021-Identification_and_Authentication_Failures/

OWASP Foundation. XML External Entity Prevention Cheat Sheet.

- https://cheatsheetseries.owasp.org/cheatsheets/XML_External_Entity_Prevention_Cheat_Sheet.html

PortSwigger. What is XXE (XML external entity) injection?

- <https://portswigger.net/web-security/xxe>